```
 auxiliar.genes[permuta[mejorCromosoma][0]] = individuo.genes[v[0]];
 auxiliar.genes[permuta[mejorCromosoma][1]] = individuo.genes[v[1]];
 auxiliar.genes[permuta[mejorCromosoma][2]] = individuo.genes[v[2]];

 for (int i = 0;i < individuo.lonCrom;i++){
             individuo.genes[i] = auxiliar.genes[i];;
      }
   }
}
```

Apéndice B

SOLUCIÓN PROYECTO 1: CÓDIGO C++

A continuación se incluye el código C++ correspondiente al Proyecto 1. Hemos seleccionado una función de cada uno de los tipos propuestos: maximización, minimización y multivarible.

Se utiliza programación estructurada y clases predefinidas de C++ como la clase **vector**.

Los módulos que se presentan son los siguientes:

- `funciones.h`: módulo con las definiciones de tipos y prototipos de funciones.
- `Funciones.cpp`: módulo con la implementación de las funciones.

La función a optimizar se especifica en la función adaptación.

El programa se puede invocar desde la línea de comandos, especificando los siguientes parámetros:

- Límite inferior del intervalo en que busca el óptimo
- Límite superior
- Tolerancia o exactitud con la que se pide el resultado
- Tamaño de población
- Número de generaciones de evolución

- Tasa de cruce
- Tasa de mutación

Por ejemplo, para la primera función que se presenta:

```
funciones 0.0 32.0 0.0001 500 1000 0.4 0.001
```

Se describen también los cambios que se introducen para la minización de una función y para trabajar con funciones de varias variables.

Módulo funciones.h

```
#include <stdlib.h>
#include <math.h>
#include <string>
#include <vector.h>

using namespace std;
// individuo
typedef vector < bool > TGenes;
typedef struct{
          TGenes genes;       // cadena de bits (genotipo)
          double x;           // fenotipo
          double aptitud;     // funcion de evaluacion
          double puntuacion; // punt. relativa:fitness/sumfitness
          double punt_acu;    // puntuacion acumulada para sorteos
          bool elite;         // indicativo de pertenecer a la elite
        }TIndividuo;
// población
typedef vector <TIndividuo> TPoblacion;


/*************************/
/* Funciones del módulo */
/*************************/
TPoblacion poblacion_inicial(int tam_pob, int lcrom, double x_min,
                             double x_max);
TIndividuo genera_indiv(int lcrom, double x_min, double x_max);
void seleccion(TPoblacion& pob, int tam_pob);
double adaptacion(TIndividuo& individuo, int lcrom, double x_min,
                  double x_max);
double decod(TGenes genes, int lcrom, double x_min, double x_max);
int bin_dec(TGenes genes, int lcrom);

void reproduccion(TPoblacion& pob, int tam_pob, int lcrom,
             double prob_cruce, double x_min, double x_max);
void cruce(TIndividuo padre1, TIndividuo padre2, TIndividuo& hijo1,
```



```
           TIndividuo& hijo2,int lcrom, int punto_cruce,
           double x_min, double x_max);
void mutacion(TPoblacion& pob, int tam_pob, int lcrom,
              double prob_mut,double x_min, double x_max);
void evaluacion(TPoblacion& pob, int tam_pob, int& pos_mejor,
                double& sumfitness);
```

Módulo funciones.cpp

Representa la función:

- $f(x) = 20 + e - 20e^{-0.2|x|} - e^{\cos 2\pi x} : x \in [0, 32]$

que presenta un máximo (22.3136) en 31.5005.

```
/*********************************************************************
/* Programacion evolutiva: proyecto 1                                */
/* version 1.0:                                                      */
/*                                                                   */
/* Modulo funciones.cpp                                              */
/*                                                                   */
/*********************************************************************

#include "funciones.h"

int main( int argc, char *argv[] )
{
  double x_min;          // límite inferior del intervalo
  double x_max;          // límite superior del intervalo
  double TOL;            // Tolerancia
  int tam_pob;           // tamaño de población
  int num_gen;           // número de generaciones
  double prob_cruce;     // probabilidad de cruce
  double prob_mut;       // probabilidad de mutación
  TPoblacion pob;        // población
  int lcrom;             // longitud de los individuos
  int generacion;        // contador de generaciones
  int pos_mejor;         // posición del mejor individuo
  double sumaptitud;     // suma de adaptaciones

  sscanf(argv[1], "%lf", &x_min ); // límite inferior del intervalo
                                   // en que se busca el óptimo
  sscanf(argv[2], "%lf", &x_max ); // límite superior del intervalo
                                   // en que se busca el óptimo
  sscanf(argv[3], "%lf", &TOL ); // tolerancia o exactitud
                                 // con que se busca el óptimo
```

```
  sscanf(argv[4], "%d", &tam_pob ); // tamaño de la población
  sscanf(argv[5], "%d", &num_gen ); // número de iteraciones
  sscanf(argv[6], "%lf", &prob_cruce ); // porcentaje de cruces
  sscanf(argv[7], "%lf", &prob_mut );  // porcentaje de mutaciones

  // inicializaciones
  lcrom = (int)ceil(log(1 + (x_max - x_min)/TOL) / log((double)2));

  // inicialización de números aleatorios
  time_t t;
  srand((unsigned) time(&t));

  // se genera la población inicial
  pob = poblacion_inicial(tam_pob, lcrom, x_min, x_max);
  // se evalúa la población
  evaluacion(pob, tam_pob, pos_mejor, sumaptitud);

  // bucle de evolución
  for (generacion=0; generacion < num_gen; generacion++){
    seleccion(pob, tam_pob);
    reproduccion(pob, tam_pob, lcrom, prob_cruce, x_min, x_max);
    mutacion(pob, tam_pob, lcrom, prob_mut, x_min, x_max);
    evaluacion(pob, tam_pob, pos_mejor, sumaptitud);
  }
  cout << "Aptitud del mejor(" << pos_mejor << ") = ";
  cout <<  pob[pos_mejor].aptitud << " en " << pob[pos_mejor].x <<
endl;
}
/*****************************************************************/
/*                                                               */
/* funcion: poblacion_inicial                                    */
/* Genera la poblacion inicial aleatoriamente                    */
/*                                                               */
/*****************************************************************/
TPoblacion poblacion_inicial(int tam_pob, int lcrom,
                             double x_min, double x_max)
{
  TPoblacion pob;
  TIndividuo indiv;
  int i;
  for(i=0; i < tam_pob; i++){
    indiv = genera_indiv(lcrom, x_min, x_max);
    pob.push_back(indiv);
  }
  return pob;
}
/*****************************************************************/
/*                                                               */
/* funcion: genera_indiv                                         */
/* Genera un individuo de la poblacion inicial                   */
/*                                                               */
/*****************************************************************/
```



```
TIndividuo genera_indiv(int lcrom, double x_min, double x_max)
{
  int i;
  TIndividuo indiv;
  int gen;

  indiv.genes.clear();
  for(i=0; i < lcrom; i++){
    if (rand() < RAND_MAX / 2 )
      gen = 0;
    else
      gen = 1;
    indiv.genes.push_back(gen);
  }
    indiv.aptitud = adaptacion(indiv, lcrom, x_min, x_max);
  return indiv;
}

/*****************************************************************/
/*                                                               */
/* funcion: seleccion                                            */
/* Selecciona un unico individuo por el metodo de la ruleta      */
/*                                                               */
/*****************************************************************/

void seleccion(TPoblacion& pob, int tam_pob)
{
  int* sel_super;                  // seleccionados para sobrevivir
  float prob,f;                    // probabilidad de selección
  int pos_super;                   // posición del superviviente
  TPoblacion pob_aux;              // población auxiliar
  int i,j;
  int TAM_ELITE = pob.size()* 2 /100;
  int* sel_elite;

  sel_super = new int[tam_pob];
  sel_elite = new int[TAM_ELITE];
  //------------------------//
  // se selecciona la elite
  //------------------------//
  for (i=0; i < TAM_ELITE; i++){
    sel_elite[i] = 0;
  }
  for (i=0; i < tam_pob; i++){
    pob[i].elite = false;
    j = 0;
    while ( (j < TAM_ELITE) &&
       (pob[sel_elite[j]].aptitud >= pob[i].aptitud)){
      j++;
    }
    if (j < TAM_ELITE){
```

```
    sel_elite[j] = i;
   }
}
for (i=0; i < TAM_ELITE; i++){
  pob[sel_elite[i]].elite = true;
}

// se seleccionan tam_pob individuos para reproducirse
// se generan números aleatorios entre 0 y 1,
// seleccionan individuos de acuerdo con su puntuación acumulada

for (i=0; i < tam_pob; i++){
  if (pob[i].elite == true)  //ELITISMO
    sel_super[i] = i;
  else{
    f = rand();
    prob = (f/(RAND_MAX+1.0));
    pos_super = 0;
    while ((prob>pob[pos_super].punt_acu)&&(pos_super < tam_pob))
      pos_super++;
    if (pos_super < tam_pob)
      sel_super[i] = pos_super;
    else
      sel_super[i] = pos_super-1;
   }
}
// se genera la población intermedia
for (i=0; i < tam_pob; i++){
  pob_aux.push_back(pob[sel_super[i]]);
}
pob.clear();
for (i=0; i < tam_pob; i++){
  pob.push_back(pob_aux[i]);
}
delete [] sel_super;
delete [] sel_elite;
}

/*****************************************************************/
/*                                                               */
/* funcion: adaptacion                                           */
/* Evalua la calidad de un individuo                             */
/*                                                               */
/*****************************************************************/


double adaptacion(TIndividuo& individuo, int lcrom,
                  double x_min, double x_max)
{
  double x; // fenotipo
  double f; // valor de la función a optimizar
```



```
  x = decod(individuo.genes, lcrom, x_min, x_max);

  individuo.x = x;

  // AQUÍ se especifica la función a optimizar
  f = 20+exp((double)1)-20*exp((double)(-0.2*fabs(x)))-
        exp((double)(cos(2*M_PI*x)));

  return f;
}


/*****************************************************************/
/*                                                               */
/* funcion: decod                                                */
/* Decodifica el genotipo de un individuo                        */
/*                                                               */
/*****************************************************************/

double decod(TGenes genes, int lcrom, double x_min, double x_max)
{
  double x;

  x=(float)bin_dec(genes, lcrom)/(pow((double)2,(double)lcrom) - 1);
  x = x_min + (x_max - x_min) * x;
  return x;
}

/*****************************************************************/
/*                                                               */
/* funcion: bin_dec                                              */
/* Convierte de binario a decimal                                */
/*                                                               */
/*****************************************************************/

int bin_dec(TGenes genes, int lcrom)
{
  int i, d=0, pot=1;

  for(i=0; i < lcrom; i++){
    d = d + pot*genes[i];
    pot = pot * 2;
  }
  return d;
}
/*****************************************************************/
/*                                                               */
/* funcion: reproduccion                                         */
/* Selecciona los individuos a reproducirse y aplica el          */
/* operador de cruce                                             */
/*                                                               */
/*****************************************************************/
```

```
void reproduccion(TPoblacion& pob, int tam_pob, int lcrom,
          double prob_cruce, double x_min, double x_max)
{
  int* sel_cruce;       //seleccionados para reproducirse
  int num_sel_cruce=0; //num de individuos seleccionados a cruzar
  double prob;
  int punto_cruce;
  TIndividuo hijo1, hijo2;
  int i;

  sel_cruce = new int[tam_pob];
  // se eligen los individuos a cruzar
  for (i=0; i < tam_pob; i++){
    // se generan tam_pob numeros aleatorios a_i en [0 1)
    prob = (rand()/(RAND_MAX+1.0));
    // se eligen para el cruce los individuos de
    // las posiciones i con a_i < prob_cruce
    if (prob < prob_cruce){
      sel_cruce[num_sel_cruce] = i;
      num_sel_cruce++;
    }
  }
  // el número de selecionados se hace par
  if ((num_sel_cruce % 2) == 1)
    num_sel_cruce--;

  // se cruzan los individuos elegidos en un punto al azar
  punto_cruce = (int)(rand() * (lcrom+1) / (RAND_MAX + 1.0)) + 1;
  for (i=0; i < num_sel_cruce; i=i+2){
    cruce(pob[sel_cruce[i]], pob[sel_cruce[i+1]], hijo1, hijo2,
          lcrom, punto_cruce, x_min, x_max);
    // los nuevos individuos sustituyen a sus progenitores
    // respetando la élite
   if ((hijo1.aptitud<adaptacion(pob[sel_cruce[i]],lcrom,x_min,
x_max)) || (!pob[sel_cruce[i]].elite)){ //ELITISMO
      pob[sel_cruce[i]] = hijo1;
    }
    if ((hijo2.aptitud <
          adaptacion(pob[sel_cruce[i+1]], lcrom, x_min, x_max))
        || (!pob[sel_cruce[i+1]].elite)){
      pob[sel_cruce[i+1]] = hijo2;
    }
  }
  delete [] sel_cruce;
}
/*****************************************************************/
/*                                                               */
/* funcion: cruce                                                */
/* Aplica el operador de cruce                                   */
/*                                                               */
/*****************************************************************/
```



```
void cruce(TIndividuo padre1, TIndividuo padre2, TIndividuo& hijo1,
TIndividuo& hijo2, int lcrom, int punto_cruce, double x_min, double
x_max)
{
  int i;
  hijo1.genes.clear();
  hijo2.genes.clear();

  // primera parte del intercambio: 1 a 1 y 2 a 2
  for(i=0; i < punto_cruce; i++){
    hijo1.genes.push_back(padre1.genes[i]);
    hijo2.genes.push_back(padre2.genes[i]);
  }
  // segunda parte: 1 a 2 y 2 a 1
  for(i=punto_cruce; i < lcrom; i++){
    hijo1.genes.push_back(padre2.genes[i]);
    hijo2.genes.push_back(padre1.genes[i]);
  }
  // se evalúan
  hijo1.aptitud = adaptacion(hijo1, lcrom, x_min, x_max);
  hijo2.aptitud = adaptacion(hijo2, lcrom, x_min, x_max);
}


/*****************************************************************/
/*                                                               */
/* funcion: mutacion                                             */
/* Aplica el operador de mutacion a la poblacion                 */
/*                                                               */
/*****************************************************************/
void mutacion(TPoblacion& pob, int tam_pob, int lcrom, double
prob_mut, double x_min, double x_max)
{
  bool mutado;
  int i,j;
  float prob;

  for (i=0; i < tam_pob; i++){
    mutado = false;
    for (j=0; j < lcrom; j++){
      // se genera un número aleatorio en [0 1)
      prob = (rand()/(RAND_MAX+1.0));
      // se mutan aquellos genes con prob < que prob_mut
      if ((prob < prob_mut) && (!pob[i].elite)){  //ELITISMO
        pob[i].genes[j] = !(bool)( pob[i].genes[j]);
        mutado = true;
      }
      if (mutado)
        pob[i].aptitud = adaptacion(pob[i], lcrom, x_min, x_max);
    }
  }
}
```

```
/*************************************************************/
/*                                                           */
/* funcion: evaluacion                                       */
/* Calcula y actualiza as estadisticas de la poblacion       */
/*                                                           */
/*************************************************************/

void evaluacion(TPoblacion& pob, int tam_pob, int& pos_mejor,
double& sumaptitud)
{
  float punt_acu = 0;     // puntación acumulada de los individuos
  float aptitud_mejor = 0;// mejor aptitud
  int i;
  sumaptitud = 0;  // suma de la aptitud

  for (i=0; i< tam_pob; i++){
    sumaptitud = sumaptitud + pob[i].aptitud;
    if (pob[i].aptitud > aptitud_mejor){
      pos_mejor = i;
      aptitud_mejor = pob[i].aptitud;
    }
  }
  for (i=0; i< tam_pob; i++){
    pob[i].puntuacion = pob[i].aptitud / sumaptitud;
    pob[i].punt_acu = pob[i].puntuacion + punt_acu;
    punt_acu = punt_acu + pob[i].puntuacion;
  }
}
```

Funciones.h para minimizar

Se introduce un nuevo campo en los individuos para almacenar la adaptación antes de transformarla para minimizar: aptitud_bruta. El resto del módulo no varía y por tanto no se incluye.

```
typedef struct{
         TGenes genes;      // cadena de bits (genotipo)
         double x;          // fenotipo
         double aptitud;    // función de adaptación
         doublé aptitud_bruta; // antes de transformarla
         double puntuacion; //punt. relativa:fitness/ sumfitness
         double punt_acu;   // puntuación acumulada para sorteos
         bool elite;        // indicativo de pertenecer a la élite
        }TIndividuo;
```


Funciones.cpp para minimizar

Representa la función:

- $f(x) = -|xsen(\sqrt{|x|})| : x \in [-250,250]$

que presenta un mínimo (-201.843) en $x = 203.814$.

El código es muy similar al presentado anteriormente. En el programa principal se incluye una llamada a la función *revisar_adap_mini*, para que realice la transformación de la adaptación bruta que proporciona la función de adaptación al valor de adaptación transformado para minimizar. Las funciones en las que se modifican individuos (*genena_indiv*, *cruce* y *mutación*) calculan la adaptación bruta, mientras que la función de selección trabaja con el valor de adaptación transformado.

```
/*************************************************************************
/*
/* Programacion evolutiva: proyecto 1                                    */
/* version 2.0:  minimización                                            */
/* Modulo funciones.cpp                                                  */
/*                                                                       */
/*************************************************************************

#include "funciones.h"


int main( int argc, char *argv[] )
{
  double x_min;          // límite inferior del intervalo
  double x_max;          // límite superior del intervalo
  double TOL;            // Tolerancia
  int tam_pob;           // tamaño de población
  int num_gen;           // número de generaciones
  double prob_cruce;     // probabilidad de cruce
  double prob_mut;       // probabilidad de mutación
  TPoblacion pob;        // población
  int lcrom;             // longitud de los individuos
  int generacion;        // contador de generaciones
  int pos_mejor;         // posición del mejor individuo
  double sumaptitud;     // suma de adaptaciones

  sscanf(argv[1], "%lf", &x_min ); // límite inferior del intervalo
                                   // en que se busca el óptimo
  sscanf(argv[2], "%lf", &x_max ); // límite superior del intervalo
                                   // en que se busca el óptimo
```

```
sscanf(argv[3], "%lf", &TOL ); // tolerancia o exactitud
                               // con que se busca el óptimo
sscanf(argv[4], "%d", &tam_pob ); // tamaño de la población
sscanf(argv[5], "%d", &num_gen ); // número de iteraciones
sscanf(argv[6], "%lf", &prob_cruce ); // porcentaje de cruces
sscanf(argv[7], "%lf", &prob_mut );  // porcentaje de mutaciones

// inicializaciones
lcrom = (int)ceil(log(1 + (x_max - x_min)/TOL) / log((double)2));

// inicialización de números aleatorios
time_t t;
srand((unsigned) time(&t));


// se genera la población inicial
pob = poblacion_inicial(tam_pob, lcrom, x_min, x_max);
// transformación de la adaptación para minimizar
revisar_adap_mini(pob, tam_pob, lcrom, cmax, x_min, x_max);

// se evalúa la población
evaluacion(pob, tam_pob, pos_mejor, sumaptitud);

// bucle de evolución
for (generacion=0; generacion < num_gen; generacion++){
  seleccion(pob, tam_pob);
  reproduccion(pob, tam_pob, lcrom, prob_cruce, x_min, x_max);
  mutacion(pob, tam_pob, lcrom, prob_mut, x_min, x_max);
  revisar_adap_mini(pob, tam_pob, lcrom, cmax, x_min, x_max);
  evaluacion(pob, tam_pob, pos_mejor, sumaptitud);
 }
 cout << "Aptitud del mejor(" << pos_mejor << ") = ";
 cout <<  pob[pos_mejor].aptitud_bruta;
 cout << " en " << pob[pos_mejor].x << endl;
}

/*************************************************************/
/*                                                           */
/* funcion: revisar_adap_mini                                */
/* Revisa la aptitud de la pob para transforma mini en maxi  */
/*                                                           */
/*************************************************************/

void revisar_adap_mini(TPoblacion& pob, int tam_pob, int lcrom,
               double& cmax, float x_min, float x_max)
{
  int i;
  cmax;

  cmax = pob[0].aptitud_bruta;
  for(i=1; i < tam_pob; i++){
    if (pob[i].aptitud > cmax)
```



```
      cmax = pob[i].aptitud_bruta;
  }
  cmax = cmax + cmax / 100;
  for(i=0; i < tam_pob; i++){
    pob[i].aptitud = cmax - pob[i].aptitud_bruta;
  }
}

/*************************************************************/
/*                                                           */
/* funcion: poblacion_inicial                                */
/* Genera la población inicial aleatoriamente                */
/*                                                           */
/*************************************************************/

TPoblacion poblacion_inicial(int tam_pob, int lcrom,
                             double x_min, double x_max)
{
  TPoblacion pob;
  TIndividuo indiv;
  int i;
  for(i=0; i < tam_pob; i++){
    indiv = genera_indiv(lcrom, x_min, x_max);
    pob.push_back(indiv);
  }
  return pob;
}

/*************************************************************/
/*                                                           */
/* funcion: genera_indiv                                     */
/* Genera un individuo de la población inicial               */
/*                                                           */
/*************************************************************/

TIndividuo genera_indiv(int lcrom, double x_min, double x_max)
{
  int i;
  TIndividuo indiv;
  int gen;

  indiv.genes.clear();
  for(i=0; i < lcrom; i++){
    if (rand() < RAND_MAX / 2 )
      gen = 0;
    else
      gen = 1;
    indiv.genes.push_back(gen);
  }
  indiv.aptitud_bruta = adaptacion(indiv, lcrom, x_min, x_max);
  return indiv;
}
```

```
/*****************************************************************/
/*                                                               */
/* funcion: seleccion                                            */
/* Selecciona un unico individuo por el metodo de la ruleta      */
/*                                                               */
/*****************************************************************/

void seleccion(TPoblacion& pob, int tam_pob)
{
  int* sel_super;                  // seleccionados para sobrevivir
  float prob,f;                    // probabilidad de selección
  int pos_super;                   // posición del superviviente
  TPoblacion pob_aux;              // población auxiliar
  int i,j;
  int TAM_ELITE = pob.size()* 2 /100;
  int* sel_elite;

  sel_super = new int[tam_pob];
  sel_elite = new int[TAM_ELITE];
  //------------------------//
  // se selecciona la élite
  //------------------------//
  for (i=0; i < TAM_ELITE; i++){
    sel_elite[i] = 0;
  }
  for (i=0; i < tam_pob; i++){
    pob[i].elite = false;
    j = 0;
    while ( (j < TAM_ELITE) &&
        (pob[sel_elite[j]].aptitud >= pob[i].aptitud)){
      j++;
    }
    if (j < TAM_ELITE){
     sel_elite[j] = i;
    }
  }
  for (i=0; i < TAM_ELITE; i++){
    pob[sel_elite[i]].elite = true;
  }

  // se seleccionan tam_pob individuos para reproducirse
  // se generan números aleatorios entre 0 y 1,
  // seleccionan individuos de acuerdo con su puntuación acumulada

  for (i=0; i < tam_pob; i++){
    if (pob[i].elite == true)  //ELITISMO
      sel_super[i] = i;
    else{
      f = rand();
      prob = (f/(RAND_MAX+1.0));
      pos_super = 0;
```



```
      while ((prob>pob[pos_super].punt_acu)&&(pos_super < tam_pob))
        pos_super++;
      if (pos_super < tam_pob)
        sel_super[i] = pos_super;
      else
        sel_super[i] = pos_super-1;
    }
  }
  // se genera la población intermedia
  for (i=0; i < tam_pob; i++){
    pob_aux.push_back(pob[sel_super[i]]);
  }
  pob.clear();
  for (i=0; i < tam_pob; i++){
    pob.push_back(pob_aux[i]);
  }
  delete [] sel_super;
  delete [] sel_elite;
}

/*****************************************************************/
/*                                                               */
/* funcion: adaptacion                                           */
/* Evalua la calidad de un individuo                             */
/*                                                               */
/*****************************************************************/

double adaptacion(TIndividuo& individuo, int lcrom,
                  double x_min, double x_max)
{
  double x; // fenotipo
  double f; // valor de la función a optimizar

  x = decod(individuo.genes, lcrom, x_min, x_max);

  individuo.x = x;

  // AQUÍ se especifica la función a optimizar
  f = -(fabs(x * sin(sqrt(fabs(x)))));

  return f;
}


/*****************************************************************/
/*                                                               */
/* funcion: decod                                                */
/* Decodifica el genotipo de un individuo                        */
/*                                                               */
/*****************************************************************/

double decod(TGenes genes, int lcrom, double x_min, double x_max)
```

```
{
  double x;

  x=(float)bin_dec(genes,lcrom)/(pow((double)2,(double)lcrom) - 1);
  x = x_min + (x_max - x_min) * x;
  return x;
}


/*****************************************************************/
/*                                                               */
/* funcion: bin_dec                                              */
/* Convierte de binario a decimal                                */
/*                                                               */
/*****************************************************************/
int bin_dec(TGenes genes, int lcrom)
{
  int i, d=0, pot=1;

  for(i=0; i < lcrom; i++){
    d = d + pot*genes[i];
    pot = pot * 2;
  }
  return d;
}


/*****************************************************************/
/*                                                               */
/* funcion: reproduccion                                         */
/* Selecciona los individuos a reproducirse y aplica el          */
/* operador de cruce                                             */
/*                                                               */
/*****************************************************************/
void reproduccion(TPoblacion& pob, int tam_pob, int lcrom,
          double prob_cruce, double x_min, double x_max)
{
  Int* sel_cruce;       // seleccionados para reproducirse
  int num_sel_cruce=0; //num de individuos seleccionados para cruzar
  double prob;
  int punto_cruce;
  TIndividuo hijo1, hijo2;
  int i;

  sel_cruce = new int[tam_pob];
  // se eligen los individuos a cruzar
  for (i=0; i < tam_pob; i++){
    // se generan tam_pob números aleatorios a_i en [0 1)
    prob = (rand()/(RAND_MAX+1.0));
    // se eligen para el cruce los individuos de
    // las posiciones i con a_i < prob_cruce
    if (prob < prob_cruce){
```



```
      sel_cruce[num_sel_cruce] = i;
      num_sel_cruce++;
    }
  }
  // el número de selecionados se hace par
  if ((num_sel_cruce % 2) == 1)
    num_sel_cruce--;

  // se cruzan los individuos elegidos en un punto al azar
  punto_cruce = (int)(rand() * (lcrom+1) / (RAND_MAX + 1.0)) + 1;
  for (i=0; i < num_sel_cruce; i=i+2){
    cruce(pob[sel_cruce[i]], pob[sel_cruce[i+1]], hijo1, hijo2,
          lcrom, punto_cruce, x_min, x_max);
    // los nuevos individuos sustituyen a sus progenitores
    // respetando la élite
    if ((hijo1.aptitud<adaptacion(pob[sel_cruce[i]],lcrom,x_min,
x_max)) || (!pob[sel_cruce[i]].elite)){ //ELITISMO
      pob[sel_cruce[i]] = hijo1;
    }
    if ((hijo2.aptitud <
          adaptacion(pob[sel_cruce[i+1]], lcrom, x_min, x_max))
        || (!pob[sel_cruce[i+1]].elite)){
      pob[sel_cruce[i+1]] = hijo2;
    }
  }
  delete [] sel_cruce;
}

/*****************************************************************/
/*                                                               */
/* funcion: cruce                                                */
/* Aplica el operador de cruce                                   */
/*                                                               */
/*****************************************************************/

void cruce(TIndividuo padre1, TIndividuo padre2, TIndividuo& hijo1,
TIndividuo& hijo2,
           int lcrom, int punto_cruce, double x_min, double x_max)
{
  int i;
  hijo1.genes.clear();
  hijo2.genes.clear();

  // primera parte del intercambio: 1 a 1 y 2 a 2
  for(i=0; i < punto_cruce; i++){
    hijo1.genes.push_back(padre1.genes[i]);
    hijo2.genes.push_back(padre2.genes[i]);
  }
  // segunda parte: 1 a 2 y 2 a 1
  for(i=punto_cruce; i < lcrom; i++){
    hijo1.genes.push_back(padre2.genes[i]);
    hijo2.genes.push_back(padre1.genes[i]);
```

```
  }
  // se evalúan
  hijo1.aptitud_bruta = adaptacion(hijo1, lcrom, x_min, x_max);
  hijo2.aptitud_bruta = adaptacion(hijo2, lcrom, x_min, x_max);
}

/*****************************************************************/
/*                                                               */
/* funcion: mutacion                                             */
/* Aplica el operador de mutacion a la población                 */
/*                                                               */
/*****************************************************************/

void mutacion(TPoblacion& pob, int tam_pob, int lcrom, double
prob_mut, double x_min, double x_max)
{
  bool mutado;
  int i,j;
  float prob;

  for (i=0; i < tam_pob; i++){
    mutado = false;
    for (j=0; j < lcrom; j++){
      // se genera un número aleatorio en [0 1)
      prob = (rand()/(RAND_MAX+1.0));
      // se mutan aquellos genes con prob < que prob_mut
      if ((prob < prob_mut) && (!pob[i].elite)){  //ELITISMO
        pob[i].genes[j] = !(bool)( pob[i].genes[j]);
        mutado = true;
      }
      if (mutado)
        pob[i].aptitud_bruta = adaptacion(pob[i], lcrom, x_min,
x_max);
    }
  }
}
/*****************************************************************/
/*                                                               */
/* funcion: evaluacion                                           */
/* Calcula y actualiza as estadisticas de la población           */
/*                                                               */
/*****************************************************************/

void evaluacion(TPoblacion& pob, int tam_pob, int& pos_mejor,
double& sumaptitud)
{
  float punt_acu = 0;     // puntacion acumulada de los individuos
  float aptitud_mejor = 0;   // mejor aptitud
  int i;
  sumaptitud = 0;  // suma de la aptitud
  for (i=0; i< tam_pob; i++){
    sumaptitud = sumaptitud + pob[i].aptitud;
```



```
    if (pob[i].aptitud > aptitud_mejor){
      pos_mejor = i;
      aptitud_mejor = pob[i].aptitud;
    }
  }
  for (i=0; i< tam_pob; i++){
    pob[i].puntuacion = pob[i].aptitud / sumaptitud;
    pob[i].punt_acu = pob[i].puntuacion + punt_acu;
    punt_acu = punt_acu + pob[i].puntuacion;
  }
}
```

Funciones.h para varias variables

Necesitamos representar ahora los genes correspondientes a cada una de las variables involucradas. Extendemos por lo tanto la representación del individuo:

```
typedef vector < bool > TGenes;      // cadena de bits (genotipo)
typedef vector< TGenes > Tvec_genes;// vector de genotipos
                                     // de cada variable
typedef vector< float > Tvec_var;    // vector de fenotipos
                                     // de cada variable
typedef vector< int > Tvec_int;      // vector para la longitudes
                                     // de los genes
typedef struct{
          Tvec_genes vec_genes;
          Tvec_var vec_x;            // fenotipo
          double aptitud_bruta; // función de evaluación
          double aptitud;     // aptitud transformada
          double puntuacion; // punt.relativa : fitness/sumfitness
          double punt_acu;   // puntuación acumulada para sorteos
        }TIndividuo;
```

Funciones.cpp para varias variables

Representa la función:

- $f(x_i \mid i = 1..n) = \sum_{i=1}^{n} -x_i sen(\sqrt{|x_i|}) : x_i \in [0,100]$

que tiene un valor de $-n$ *63.63498 y se encuentra en $x_i = 65.54785$.

Las funciones se modifican sólo para aplicarse a todas las cadenas de genes incluidas en la representación. Incluimos aquí sólo las funciones que se ven afectadas:

```
/*************************************************************/
/*                                                           */
/* funcion: genera_indiv                                     */
/* Genera un individuo de la población inicial               */
/*                                                           */
/*************************************************************/

TIndividuo genera_indiv(Tvac_int lcrom, Tvec_var x_min, Tvec_var
x_max)
{
  int i,j;
  TIndividuo indiv;
  TGenes genes;
  int gen;
  float f;

  indiv.vec_genes.clear();
  /*-------------------------------------------*/
  /* se generan los genotipos para cada variable */
  /*-------------------------------------------*/
  for(i=0; i< NUM_VAR; i++){
    genes.clear();
    for(j=0; j < lcrom[i]; j++){
      f = rand();
      if (f < RAND_MAX / 2 )
        gen = 0;
      else
        gen = 1;
      genes.push_back(gen);
    }
    indiv.vec_genes.push_back(genes);
  }
  indiv.aptitud_bruta = adaptacion(indiv, lcrom, x_min, x_max);
  return indiv;
}
/*************************************************************/
/*                                                           */
/* funcion: adaptacion                                       */
/* Evalua la calidad de un individuo                         */
/*                                                           */
/*************************************************************/

float adaptacion(TIndividuo& individuo, Tvec_int lcrom,
                 Tvec_var x_min, Tvec_var x_max)
{
  int i;
  Tvec_var x; // vector de fenotipos
```



```
  float f; // valor de la funcion a optimizar
  x.clear();
  individuo.vec_x.clear();
  for(i=0; i < NUM_VAR; i++){
    individuo.vec_x.push_back(
      decod(individuo.vec_genes[i], lcrom[i], x_min[i], x_max[i]));
  }
  x = individuo.vec_x;

  f = 0;
  for(i=0; i < NUM_VAR; i++){
    f = f - x[i] * sin(sqrt(fabs(x[i])));
  }
  return f;
}
/*************************************************************/
/*                                                           */
/* funcion: cruce                                            */
/* Aplica el operador de cruce                               */
/*                                                           */
/*************************************************************/

void cruce(TIndividuo padre1, TIndividuo padre2, TIndividuo& hijo1,
       TIndividuo& hijo2,Tvec_int lcrom, Tvec_int punto_cruce,
       Tvec_var x_min, Tvec_var x_max)
{
  int i,j;
  TGenes genes_hijo1, genes_hijo2;

  hijo1.vec_genes.clear();
  hijo2.vec_genes.clear();

  for(j=0; j < NUM_VAR; j++){
    // primera parte del intercambio: 1 a 1 y 2 a 2
    genes_hijo1.clear();
    genes_hijo2.clear();
    for(i=0; i < punto_cruce[j]; i++){
      genes_hijo1.push_back(padre1.vec_genes[j][i]);
      genes_hijo2.push_back(padre2.vec_genes[j][i]);
    }
    // segunda parte: 1 a 2 y 2 a 1
    for(i=punto_cruce[j]; i < lcrom[j]; i++){
      genes_hijo1.push_back(padre2.vec_genes[j][i]);
      genes_hijo2.push_back(padre1.vec_genes[j][i]);
    }
    hijo1.vec_genes.push_back(genes_hijo1);
    hijo2.vec_genes.push_back(genes_hijo2);
  }
  // se evalúan
  hijo1.aptitud_bruta = adaptacion(hijo1, lcrom, x_min, x_max);
  hijo2.aptitud_bruta = adaptacion(hijo2, lcrom, x_min, x_max);
}
```

```
/*************************************************************/
/*                                                           */
/* funcion: mutacion                                         */
/* Aplica el operador de mutacion a la población             */
/*                                                           */
/*************************************************************/
void mutacion(TPoblacion& pob, int tam_pob, Tvec_int lcrom,
              float prob_mut, Tvec_var x_min, Tvec_var x_max )
{
  bool mutado;
  int i,j,k;
  float prob,f;

  for (i=0; i < tam_pob; i++){
    mutado = false;
    for(k=0; k <NUM_VAR; k++){
      for (j=0; j < lcrom[k]; j++){
        // se genera un número aleatorio en [0 1)
        f = rand();
        prob = (f/(RAND_MAX+1.0));
        // se mutan aquellos genes con prob < que prob_mut
        if ((prob < prob_mut) && (!pob[i].elite)){
          pob[i].vec_genes[k][j] = !(bool)( pob[i].vec_genes[k][j]);
          mutado = true;
        }
      }
    }
    if (mutado)
     pob[i].aptitud_bruta = adaptacion(pob[i], lcrom, x_min, x_max);
  }
}
```

BIBLIOGRAFÍA

[ALB01] Alba, E., Troya, J.M. *Analyzing synchronous and asynchronous parallel distributed genetic algorithms.* Future Generation Comp. Syst., 17(4), pág. 451-465, (2001).

[ALB02] Alba, E., Tomassini, M. *Parallelism and evolutionary algorithms*. IEEE Trans. Evolutionary Computation, 6(5), pág. 443-462, (2002).

[ANC03] Anchor, K.P., Zydallis, J.B., Gunsch, G.H., Lamont, G.B. *Different Multi-objective Evolutionary Programming Approaches for Detecting Computer Network Attacks.* Proc. Evolutionary Multi-Criterion Optimization. Second International Conference, EMO 2003, Springer. Lecture Notes in Computer Science. Volume 2632, pág. 707-721, (2003).

[APP06] Applegate, D. L.; Bixby, R. M.; Chvátal, V. & Cook. *The Traveling Salesman Problem*. Princeton Series in applied Mathematics, (2006).

[BAC91] Bäck, T., Hoffmeister, F. Schwefel, H.P.. *A survey of evolution strategies.* Proc. of the Fourth International Conference on Genetic Algorithms, Morgan Kaufman, pág. 2-9, (1991).

[BAC94] Bäck, T. *Selective pressure in evolutionary algorithms: A characterization of selection mechanisms.* Proc. of the First IEEE Conf. on Evolutionary Computation, IEEE Press, pág. 57-62, (1994).

[BAC96A] Bäck, T.. *Evolutionary Algorithms in Theory and Practice*. Oxford University Press, New York, (1996).